COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/04/2026	**DATE SUBMITTED:**	09/09/2026
NAME:	Libo-on, Gian Carlo G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. **SORTING ALGORITHMS**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int SIZE = 100;

void displayArray(int arr[]);

int main() {

    // Create an array with unsorted list with 100 maximum size
    int unsortedArr[SIZE];

    // Use time(0) as seed
    std::srand(std::time(0));

    // Populate the array
    for (int i = 0; i < SIZE; i++) {
        unsortedArr[i] = std::rand() % 100 + 1;  // Random numbers between 1 and 100
    }
    std::cout << "Unsorted Array: " << std::endl;
    // Display the unsorted array
    displayArray(unsortedArr);
```

```cpp
void displayArray(int arr[]) {
    for (int i = 0; i < SIZE; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}
```

```
Unsorted Array:
37 61 95 11 45 8 61 94 58 28 25 72 80 49 61 73 25 60 46 59 90 65 40 2
 43 39 60 82 64 10 72 56 3 79 99 94 6 53 48 70 95 77 5 80 70 39 17 6

92 30 29 77 24 91 81 79 95 95 68 55 45 34 93 34 24 99 5 43 67 24 23 73 31 19 58 19 2
87 19 70 78 60 22 41 43 8 83 2 47 77 23 36 28 72 55 44 66 99 33 64
```

Observations	Basically, the code initializes an array with a max size of 100 items. These items are randomly generated from values between 1 - 100. Then it displays the values generated.

Table 7-1. Array of Values for Sort Algorithm Testing

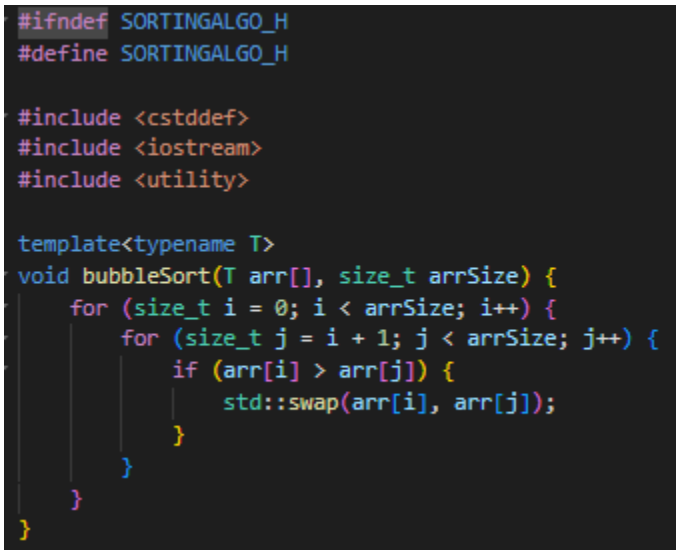
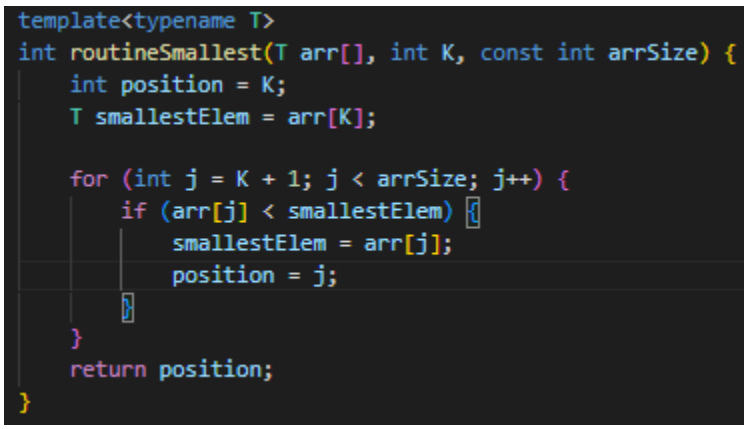
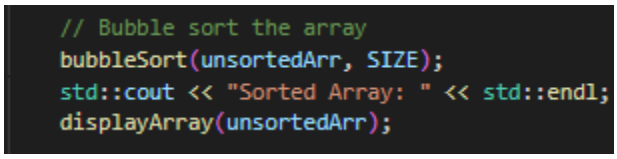
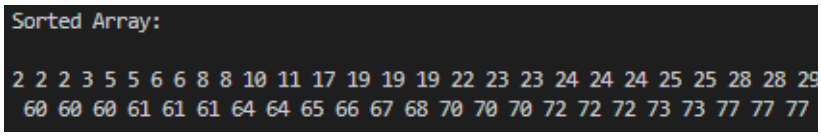
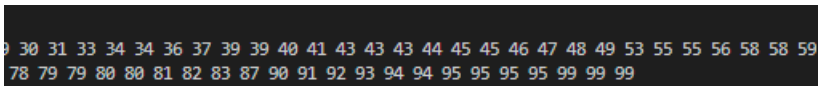
Code + Console Screenshot	    
Observations	The h file creates a function that can be called in the main file. The functions created resemble a bubble sorting technique which compares the value of i and j for each iteration and switches them if the value of the preceding index is greater than the next.

Table 7-2. Bubble Sort Technique

```
#ifndef SORTINGALGO_H
#define SORTINGALGO_H

#include <cstddef>
#include <iostream>
#include <utility>

template<typename T>
void bubbleSort(T arr[], size_t arrSize) {
    for (size_t i = 0; i < arrSize; i++) {
        for (size_t j = i + 1; j < arrSize; j++) {
            if (arr[i] > arr[j]) {
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

```
template<typename T>
int routineSmallest(T arr[], int K, const int arrSize) {
    int position = K;
    T smallestElem = arr[K];

    for (int j = K + 1; j < arrSize; j++) {
        if (arr[j] < smallestElem) {
            smallestElem = arr[j];
            position = j;
        }
    }
    return position;
}
```

```
    // Bubble sort the array
    bubbleSort(unsortedArr, SIZE);
    std::cout << "Sorted Array: " << std::endl;
    displayArray(unsortedArr);
```

```
Sorted Array:

2 2 2 3 5 5 6 6 8 8 10 11 17 19 19 19 22 23 23 24 24 24 25 25 28 28 29
 60 60 60 61 61 61 64 64 65 66 67 68 70 70 70 72 72 72 73 73 77 77 77
```

```
9 30 31 33 34 34 36 37 39 39 40 41 43 43 43 44 45 45 46 47 48 49 53 55 55 56 58 58 59
78 79 79 80 80 81 82 83 87 90 91 92 93 94 94 95 95 95 95 99 99 99
```

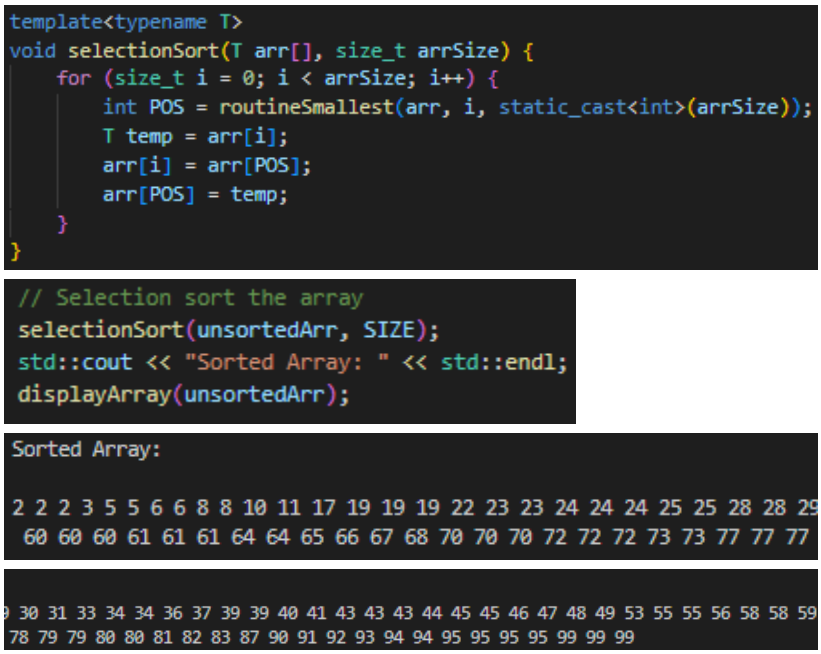
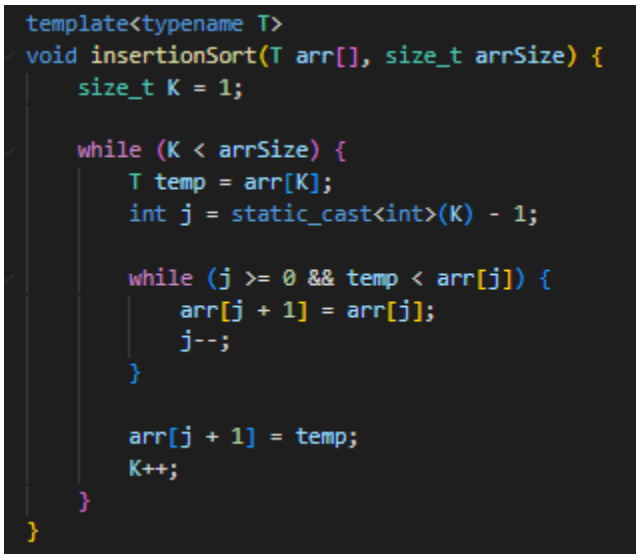
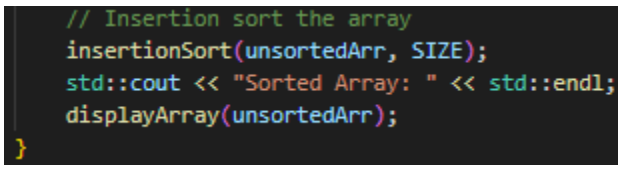
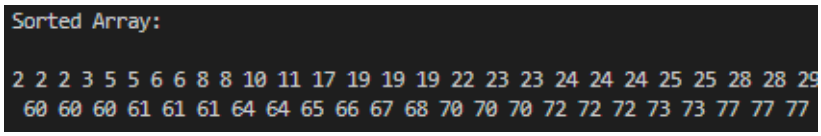
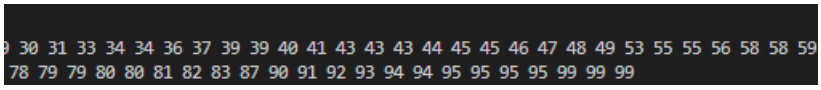
Code + Console Screenshot	
Observations	The selection sort iterates through the entire unsorted data and detects the smallest value i. Then i is assigned to a temporary index. That temporary index is then moved into the front of the array then the next iteration starts at the right of the sorted index.. It repeats until everything is in place.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	   
Observations	The insertion technique compares the items in the indexes one by one until all the items in the unsorted data are inserted into a new sorted array. It repeats until all items is inserted in the sorted array.

```
template<typename T>
void insertionSort(T arr[], size_t arrSize) {
    size_t K = 1;

    while (K < arrSize) {
        T temp = arr[K];
        int j = static_cast<int>(K) - 1;

        while (j >= 0 && temp < arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
        K++;
    }
}
```

```
    // Insertion sort the array
    insertionSort(unsortedArr, SIZE);
    std::cout << "Sorted Array: " << std::endl;
    displayArray(unsortedArr);
}
```

```
Sorted Array:

2 2 2 3 5 5 6 6 8 8 10 11 17 19 19 19 22 23 23 24 24 24 25 25 28 28 29
 60 60 60 61 61 61 64 64 65 66 67 68 70 70 70 72 72 72 73 73 77 77 77
```

```
9 30 31 33 34 34 36 37 39 39 40 41 43 43 43 44 45 45 46 47 48 49 53 55 55 56 58 58 59
78 79 79 80 80 81 82 83 87 90 91 92 93 94 94 95 95 95 95 99 99 99
```

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode

```
Start Main Program
    Set constant max size to 100
    Initialize candidate names
    Set originalVotes = Array of size SIZE
    Set originalVotes = testVotes
    Create Random Value Generator
    Print Unsorted Array
    Call display array

    //bubble sort
    COPY originalVotes TO testVotes
    start = GET_CURRENT_TIME()
    CALL bubbleSort(testVotes, SIZE)
    stop = GET_CURRENT_TIME()
    PRINT formatted results

    //selection sort
    COPY originalVotes TO testVotes
    start = GET_CURRENT_TIME()
    CALL selectionSort(testVotes, SIZE)
    stop = GET_CURRENT_TIME()
    PRINT formatted results

    //insertion sort
    COPY originalVotes TO testVotes
    start = GET_CURRENT_TIME()
    CALL insertionSort(testVotes, SIZE)
    stop = GET_CURRENT_TIME()
    PRINT formatted results

End Program

Helper Functions
START
START FUNCTION displayArray():
    FOR EACH element IN arr DO
        PRINT element
    END FOR
END FUNCTION

 countAndPrintVotes()
    voteCounts = Array of size 5 initialized to 0

    // Tally vote
    FOR EACH vote IN votes DO
```

```
      voteCounts[vote - 1] = voteCounts[vote - 1] + 1
   END FOR

   maxVotes = -1
   winnerIndex = 0

   // Display results and find the candidate with the highest vote count
   FOR i = 0 TO 4 DO
      PRINT Results
      IF voteCounts[i] > maxVotes THEN
         maxVotes = voteCounts[i]
         winnerIndex = i
      END IF
   END FOR

   PRINT results
   PRINT "WINNER"
END

INITIALIZE HEADER FILE

// 1BUBBLE SORT
FUNCTION bubbleSort(arr, size):
   FOR i = 0 TO size - 2 DO
      swapped = FALSE
      FOR j = 0 TO size - i - 2 DO
         IF arr[j] > arr[j + 1] THEN
            SWAP arr[j] AND arr[j + 1]
            swapped = TRUE
         END IF
      END FOR
      IF swapped IS FALSE THEN
         BREAK if Array is already sorted
      END IF
   END FOR
END FUNCTION

// SELECTION SORT
FUNCTION selectionSort(arr, size):
   FOR i = 0 TO size - 2 DO
      minIndex = i
      FOR j = i + 1 TO size - 1 DO
         IF arr[j] < arr[minIndex] THEN
            minIndex = j
         END IF
      END FOR
      IF minIndex != i THEN
         SWAP arr[i] AND arr[minIndex]
```

```
    END IF
  END FOR
END FUNCTION

// INSERTION SORT
FUNCTION insertionSort(arr, size):
  FOR i = 1 TO size - 1 DO
    key = arr[i]
    j = i - 1
    WHILE j >= 0 AND arr[j] > key DO
      arr[j + 1] = arr[j]
      j = j - 1
    END WHILE
    arr[j + 1] = key
  END FOR
END FUNCTION
```

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>
#include <chrono>
#include <algorithm>
#include "supplementary.h"

const int SIZE = 100;

void displayArray(const int arr[]) {
    for (int i = 0; i < SIZE; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << "\n";
}

void countAndPrintVotes(const int votes[], const std::string candidates[]) {
    int voteCounts[5] = {0};

    //counts the occurance of numbers of candidates
    for (int i = 0; i < SIZE; i++) {
        voteCounts[votes[i] - 1]++;
    }

    //winner detector
    int maxVotes = -1;
    int winnerIndex = 0;

    //candidate display function with their specific numbers and votes
    for (int i = 0; i < 5; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << candidates[i] << "): "
                  << voteCounts[i] << " votes\n";
        if (voteCounts[i] > maxVotes) {
            maxVotes = voteCounts[i];
            winnerIndex = i;
        }
    }

    std::cout << "-> WINNER: " << candidates[winnerIndex]
              << " with " << maxVotes << " votes!\n";
}
```

Figure 1

Displays the array size using iterations and basic printing. Count and print function counts the appearance of the candidate number and increments every time the number of a certain candidate appears. Next is the display function, basically displays the results of the voting in a formatted manner.

```
//candidates list
int main() {
    std::string candidates[5] = {
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };

    int originalVotes[SIZE];
    int testVotes[SIZE];

    //random number generator from 1-5
    std::srand(static_cast<unsigned int>(std::time(0)));
    for (int i = 0; i < SIZE; i++) {
        originalVotes[i] = std::rand() % 5 + 1;
    }
    std::cout << "-------ELECTION VOTE TALLY -------\n\n";
    std::cout << "----------------------------------------\n";
    std::cout << "UNSORTED VOTES:\n";
    displayArray(originalVotes);
    std::cout << "----------------------------------------\n\n";

    //BUBBLE SORT
    std::copy(originalVotes, originalVotes + SIZE, testVotes);
    auto start = std::chrono::high_resolution_clock::now();

    bubbleSort(testVotes, SIZE);
    //checks the execution time of the sorting technique
    auto stop = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(stop - start);

    //displays the result and execution time
    std::cout << "1. BUBBLE SORT:\n";
    countAndPrintVotes(testVotes, candidates);
    std::cout << "   Execution Time: " << duration.count() << " microseconds\n\n";


    //SELECTION SORT
    std::copy(originalVotes, originalVotes + SIZE, testVotes);
    start = std::chrono::high_resolution_clock::now();

    selectionSort(testVotes, SIZE);
    //checks the execution time of the sorting technique
    stop = std::chrono::high_resolution_clock::now();
    duration = std::chrono::duration_cast<std::chrono::microseconds>(stop - start);

    //displays the result and execution time
    std::cout << "2. SELECTION SORT:\n";
    countAndPrintVotes(testVotes, candidates);
    std::cout << "   Execution Time: " << duration.count() << " microseconds\n\n";
```

Figure 2.

```
    //INSERTION SORT
    std::copy(originalVotes, originalVotes + SIZE, testVotes);
    start = std::chrono::high_resolution_clock::now();

    insertionSort(testVotes, SIZE);
    //checks the execution time of the sorting technique
    stop = std::chrono::high_resolution_clock::now();
    duration = std::chrono::duration_cast<std::chrono::microseconds>(stop - start);

    //displays the result and execution time
    std::cout << "3. INSERTION SORT:\n";
    countAndPrintVotes(testVotes, candidates);
    std::cout << "   Execution Time: " << duration.count() << " microseconds\n\n";

    return 0;
}
```

Figure 3.

Figure 2 and 3 contains the dictionary for candidates. Random values are also generated in using the srand function. Then the data is displayed in a formatted manner. By calling the function of the bubble sort, selection sort and insertion sort, their void functions in the h file are initiated and done by the system. To calculate and display the execution time, a timer is created using the ctime library which automatically detects the runtime from start to finish.

```
#ifndef SUPPLEMENTARY_H
#define SUPPLEMENTARY_H

#include <algorithm>

// 1. BUBBLE SORT
void bubbleSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) {
            break;
        }
    }
}
```

Figure 4

Figure 4 is a bubble sort code snippet with a bit of optimization. The optimization initiates if the data is already sorted then the sorting will stop.

```cpp
// 2. SELECTION SORT
void selectionSort(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i], arr[minIndex]);
        }
    }
}
```

Figure 5.

Figure 5 is a code snippet for Selection sort. Again the optimization includes checking if the data is already sorted.

```cpp
// 3. INSERTION SORT
void insertionSort(int arr[], int size) {
    for (int i = 1; i < size; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

#endif // SORTING_ALGO_H
```

Figure 6.

Figure 6 is a code snippet for insertion sort. Again the optimization includes checking if the data is already sorted.

```
1. BUBBLE SORT:
Candidate 1 (Bo Dalton Capistrano): 23 votes
Candidate 2 (Cornelius Raymon Agustin): 20 votes
Candidate 3 (Deja Jayla Banaga): 17 votes
Candidate 4 (Lalla Brielle Yabut): 19 votes
Candidate 5 (Franklin Relano Castro): 21 votes
 WINNER: Bo Dalton Capistrano with 23 votes!
 Execution Time: 13 microseconds
```

Figure 7. Bubble sort result

```
2. SELECTION SORT:
Candidate 1 (Bo Dalton Capistrano): 23 votes
Candidate 2 (Cornelius Raymon Agustin): 20 votes
Candidate 3 (Deja Jayla Banaga): 17 votes
Candidate 4 (Lalla Brielle Yabut): 19 votes
Candidate 5 (Franklin Relano Castro): 21 votes
 WINNER: Bo Dalton Capistrano with 23 votes!
 Execution Time: 9 microseconds
```

Figure 8. Selection sort result

```
3. INSERTION SORT:
Candidate 1 (Bo Dalton Capistrano): 23 votes
Candidate 2 (Cornelius Raymon Agustin): 20 votes
Candidate 3 (Deja Jayla Banaga): 17 votes
Candidate 4 (Lalla Brielle Yabut): 19 votes
Candidate 5 (Franklin Relano Castro): 21 votes
 WINNER: Bo Dalton Capistrano with 23 votes!
 Execution Time: 4 microseconds
```

Figure 8. Insertion sort result

Analysis:

The insertion sort is the best sorting algorithm to use in this data set. This is because it ran the fastest runtime with 4 microseconds only. The worst possible time complexity for all the sorting techniques was $O(n^2)$. However, the best case time complexity was achieved by the insertion sort because it linearly compared every data in the array making the time complexity O(n). Although bubble sort achieved the same feat, it runs much slower than the insertion result which is why insertion sort is the best technique used in this data set.

Conclusion:

In this activity, I learned that sorting algorithms play a crucial role in how data can be processed. And in this voting tally activity, I have come to realize that there are some advantages and disadvantages between each sorting algorithm. This shows in their runtime or execution time difference. In sorting or processing larger data, this execution time is much more noticeable which is why faster runtimes are preferred by the programmers. Overall, I've learned how to analyze and justify which is a better technique in solving or processing a data set.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Pseudocode assistance and idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I only asked how to do a pseudocode properly and I followed the steps myself.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.